

3. Marco Teórico

3.1 Model-Driven Engineering (MDE)

Model-Driven Engineering (MDE) es un paradigma de la ingeniería de software que posiciona a los modelos como artefactos de primera clase dentro del proceso de desarrollo, es decir, como elementos centrales y ejecutables, y no como simples documentos de apoyo. En este enfoque, el modelo se convierte en la fuente autoritativa del sistema, y las implementaciones se derivan de él mediante transformaciones automatizadas. Los modelos se definen con conceptos cercanos al dominio del problema, facilitando su comprensión y mantenimiento [1].

MDE busca aumentar la productividad y la calidad del software mediante la creación sistemática de modelos. Según estudios recientes, MDE ha demostrado ser efectivo en múltiples dominios: el 56.8% de los estudios se centran en desarrollo general, el 21.6% en sistemas embebidos, y el 13.5% en aplicaciones web. A pesar de sus beneficios, su adopción enfrenta barreras como la falta de conocimiento y habilidades, y las dificultades con las herramientas disponibles [1].

3.2 Model-Driven Architecture (MDA) — CIM/PIM/PSM

Model-Driven Architecture (MDA) es una iniciativa del Object Management Group (OMG) que estructura el desarrollo en tres niveles de abstracción:

Nivel	Descripción
CIM (Computation Independent Model)	Modelo de requisitos y contexto del negocio, sin detalles de implementación. Usa BPMN [2].
PIM (Platform Independent Model)	Estructura y funcionalidad del sistema de manera abstracta. Usa UML [2].
PSM (Platform Specific Model)	Información técnica de implementación en una plataforma particular (Java/Spring) [2].

Los requisitos del CIM deben ser trazables hasta el PIM y PSM. Las transformaciones se realizan en dos etapas: CIM→PIM y PIM→PSM. La transformación CIM→PIM es menos frecuente debido a las diferencias fundamentales entre ambos niveles [2].

3.3 Model-Driven Development (MDD)

Model-Driven Development (MDD) es la aplicación práctica de MDE, donde los modelos guían la construcción del sistema y el código se genera automáticamente. MDD mejora la productividad y la calidad del software, especialmente en proyectos complejos. Permite que expertos del dominio participen en el desarrollo mediante abstracciones de alto nivel [3].

3.4 Transformaciones M2M

Las transformaciones Model-to-Model (M2M) producen un modelo destino a partir de un modelo origen [4]. Pueden ser:

Tipo	Dirección
Horizontal	Mismo nivel de abstracción
Vertical	Diferente nivel de abstracción (ej. PIM→PSM)

El estándar OMG es QVT. Otros lenguajes incluyen ATL y EGL. La calidad de estas transformaciones es crítica para la fidelidad de los modelos generados. Los desafíos incluyen el manejo de colecciones, recursión y condiciones lógicas [4].

3.5 Transformaciones M2T

Las transformaciones Model-to-Text (M2T) producen código fuente a partir de un modelo. El estándar OMG es MOFM2T, implementado por Acceleo. Otros lenguajes incluyen Xtend y EJS (usado por StarUML) [5].

Lenguaje	Estado	Características
Acceleo	Mantenimiento pasivo	Implementa MOFM2T
Xtend	Futuro incierto	Tipado estático, integración Java
EJS	Activo	Basado en JavaScript, usado por StarUML

3.6 Metamodelo

Un metamodelo define la sintaxis abstracta de un lenguaje de modelado. UML se define sobre MOF [5]. La jerarquía de cuatro capas:

Capa	Descripción	Ejemplo
M3	Meta-metamodelo	MOF
M2	Metamodelo	Metamodelo UML
M1	Modelo	Diagrama de clases de MundiPets
M0	Instancias	Objetos en ejecución

Ecore es la herramienta líder (70.3% de los estudios). La composición de modelos se clasifica en: basada en operador, basada en aspectos y basada en reglas [6].

3.7 Perfiles UML

Los Perfiles UML extienden el metamodelo UML sin modificarlo, mediante:

Mecanismo	Descripción
Estereotipo	Nueva variante de elemento UML
Valor etiquetado	Propiedades adicionales

Restricción	Reglas OCL
Extensión de metaclasses	Extiende metaclasses específicas

Los perfiles más comunes se enfocan en sistemas embebidos, arquitectura empresarial, seguridad y desarrollo web [5].

3.8 Plantilla / Template

Las plantillas permiten la reutilización y parametrización de modelos [7]. Existen dos tipos:

Tipo	Descripción
Plantillas de modelo	Modelos parametrizados que capturan patrones estructurales
Plantillas de transformación (M2T)	Fragmentos de texto parametrizados para generar código

Las plantillas M2T combinan: (1) texto fijo, (2) instrucciones de control, y (3) referencias al modelo [8].

Herramienta	Lenguaje de plantillas
StarUML	EJS
Eclipse/Accelo	MOFM2T
Enterprise Architect	Plantillas propietarias
Visual Paradigm	Template Editor

3.9 Forward, Reverse y Round-Trip Engineering

Concepto	Descripción
Forward Engineering	Generación de código a partir del modelo
Reverse Engineering	Creación de modelo a partir del código existente
Round-Trip Engineering (RTE)	Sincronización bidireccional entre modelo y código

El RTE requiere: (1) preservación de información, (2) sincronización bidireccional, y (3) mecanismos de actualización (instantánea o bajo demanda). Existen bloques protegidos para preservar código escrito a mano durante la regeneración [9], [10].

3.10 Diagramas UML Admisibles como Origen

Diagrama	Obligación	Uso
Clases	Obligatorio	Base para generación estructural [9], [11].

Estados	Recomendado	Máquinas de estados (patrón State) [9], [11].
Secuencia	Recomendado	Interacciones entre objetos [9], [11].
Actividades	Recomendado	Flujos de trabajo y lógica de negocio [9], [11].

3.11 Estado del Arte

1. Adopción industrial

Los principales desafíos en MDE son: escalabilidad, interoperabilidad, integración con metodologías ágiles y evolución de lenguajes. El 80% de los enfoques se aplican a ejemplos académicos, no industriales [1].

2. Integración con IA y Machine Learning

Técnica	Aplicación
NLP	Automatización del análisis de requisitos [2].
ML	Generación de modelos y clasificación [2].
LLM	Asistencia en desarrollo de transformaciones [2].

ModelSet contiene 5,466 metamodelos Ecore y 5,120 modelos UML para entrenar modelos de ML [2].

3. Desafíos en Transformaciones

Desafío	Descripción
Complejidad	Manejo de colecciones, recursión y condiciones [12].
Detección de errores	Falta de técnicas automáticas [12].
Oráculos de prueba	Dificultad para determinar resultados esperados [12].
Debugging	Limitado soporte para depuración paso a paso [12].

4. Herramientas

StarUML: CLI para generación automatizada con EJS [13].

Enterprise Architect: Plantillas personalizables [13].

Visual Paradigm: Round-trip engineering [13].

Eclipse Papyrus + Acceleo: Estándares OMG [13].

Referencias

- [1] A. Bucchiarone, J. Cabot, R. F. Paige, and A. Pierantonio, “Grand challenges in model-driven engineering: an analysis of the state of the research,” *Softw. Syst. Model.*, vol. 19, no. 1, pp. 5–13, Jan. 2020, doi: 10.1007/s10270-019-00773-6.
- [2] Z. Zammel, M. Rekik, L. Souifi, and I. Rodriguez, “Towards AI-Enabled Model-Driven Architecture: Systematic Literature Review,” in *Proceedings of the 17th International Conference on Agents and Artificial Intelligence*, SCITEPRESS - Science and Technology Publications, 2025, pp. 1380–1387. doi: 10.5220/0013374000003890.
- [3] Á. Domingo, J. Echeverría, Ó. Pastor, and C. Cetina, “Evaluating the Benefits of Model-Driven Development,” 2020, pp. 353–367. doi: 10.1007/978-3-030-49435-3_22.
- [4] J. Troya, S. Segura, L. Burgueño, and M. Wimmer, “Model Transformation Testing and Debugging: A Survey,” *ACM Comput. Surv.*, vol. 55, no. 4, pp. 1–39, Apr. 2023, doi: 10.1145/3523056.
- [5] A. Zafar, F. Azam, A. Latif, M. W. Anwar, and A. Safdar, “Exploring the Effectiveness and Trends of Domain-Specific Model Driven Engineering: A Systematic Literature Review (SLR),” *IEEE Access*, vol. 12, pp. 86809–86830, 2024, doi: 10.1109/ACCESS.2024.3414503.
- [6] A. Abouzahra, A. Sabraoui, and K. Afdel, “Model composition in Model Driven Engineering: A systematic literature review,” *Inf. Softw. Technol.*, vol. 125, p. 106316, Sep. 2020, doi: 10.1016/j.infsof.2020.106316.
- [7] G. Vanwormhoudt, M. Allon, O. Caron, and B. Carré, “Template based model engineering in UML,” in *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*, New York, NY, USA: ACM, Oct. 2020, pp. 47–56. doi: 10.1145/3365438.3410988.
- [8] X. He and T. Zan, “BIT: A template-based approach to incremental and bidirectional model-to-text transformation,” *Journal of Systems and Software*, vol. 216, p. 112148, Oct. 2024, doi: 10.1016/j.jss.2024.112148.
- [9] D. Rosca and L. Domingues, “A Systematic Comparison of Roundtrip Software Engineering Approaches applied to UML Class Diagram,” *Procedia Comput. Sci.*, vol. 181, pp. 861–868, 2021, doi: 10.1016/j.procs.2021.01.240.
- [10] S. Almutairi, A. Zolotas, and D. Kolovos, “Towards round-Trip engineering of code fragments embedded in models,” in *Proceedings - ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems, MODELS 2022: Companion Proceedings*, Association for Computing Machinery, Inc, Oct. 2022, pp. 529–538. doi: 10.1145/3550356.3561578.
- [11] Y.-H. Kim, “Automatic Code Model Generation for State Machine with Internal Transitions,” *Journal of the Korea Institute of Information and Communication Engineering*, vol. 28, no. 7, pp. 841–849, Jul. 2024, doi: 10.6109/jkiice.2024.28.7.841.
- [12] J. A. H. López, J. L. Cánovas Izquierdo, and J. S. Cuadrado, “ModelSet: a dataset for machine learning in model-driven engineering,” *Softw. Syst. Model.*, vol. 21, no. 3, pp. 967–986, Jun. 2022, doi: 10.1007/s10270-021-00929-3.
- [13] A. C. Marcén, A. Iglesias, R. Lapeña, F. Pérez, and C. Cetina, “A Systematic Literature Review of Model-Driven Engineering Using Machine Learning,” *IEEE Transactions on Software Engineering*, vol. 50, no. 9, pp. 2269–2293, Sep. 2024, doi: 10.1109/TSE.2024.3430514.